

End-to-End Compiler for Custom DSL – Explanation Document

RA2311026050273 – Priya Dharshini [CSE AIML D]

1. Problem Understanding

This project involves designing a complete compiler for a custom Domain-Specific Language (DSL). The compiler processes source code through multiple stages such as lexical analysis, syntax analysis, semantic analysis, and code generation. The final output is low-level code using LLVM Intermediate Representation (IR), which can be converted into executable machine code.

2. Logic Used

The compiler is designed in a modular way with the following phases:

1. Lexical Analysis (Lexer)

- Converts source code into tokens
- Identifies keywords, identifiers, operators, and literals

2. Syntax Analysis (Parser)

- Checks whether the input follows grammar rules
- Builds a parse tree / Abstract Syntax Tree (AST)

3. AST Construction

- Represents program structure in hierarchical form
- Simplifies further processing

4. Semantic Analysis

- Checks for errors like undeclared variables, type mismatch
- Ensures logical correctness of the program

5. Intermediate Code Generation (LLVM IR)

- Converts AST into LLVM Intermediate Representation
- Generates low-level instructions

6. Code Generation

- LLVM IR is compiled using llc to produce machine code

3. Justification of Design

- **Modular Design** improves readability and scalability
- **LLVM IR** ensures portability across different platforms
- **AST Representation** simplifies optimization and code generation
- Each phase performs a specific task, making debugging easier

4. Sample Input (DSL Code)

```
let x = 10  
let y = 20  
let z = x + y  
print z
```

5. Sample Output

Tokens (Lexical Analysis):

```
LET IDENTIFIER(x) = NUMBER(10)
LET IDENTIFIER(y) = NUMBER(20)
LET IDENTIFIER(z) = IDENTIFIER(x) + IDENTIFIER(y)
PRINT IDENTIFIER(z)
```

AST Representation:

```
      =
    /\
  z  +
    /\
  x  y
```

LLVM IR (Intermediate Code):

```
%1 = add i32 10, 20
call void @print(i32 %1)
```

Final Output:

```
30
```

6. Result Interpretation

- The compiler successfully processes DSL code
- Converts high-level instructions into low-level representation
- Produces correct output after execution

7. Conclusion

This project demonstrates the complete workflow of a compiler from source code to executable output. By integrating lexing, parsing, semantic analysis, and LLVM-based code generation, it provides a scalable and efficient compilation pipeline.